

SHIELD: An Explainable Machine Learning Framework for Mule Account Detection and Investigator Workflow Automation

Rakshit Patil

Electronic and computer Engineering
PICT
Pune, India

Tarun Patil

Information Technology
PICT
Pune, India

Anuj Jadav

Information Technology
PICT
Pune, India

Pranav Patil

Electronic and computer Engineering
PICT
Pune, India

Abstract—Mule accounts are the primary conduit through which fraudulent and laundered funds re-enter the formal banking system, yet most institutions still detect them only after a suspicious activity report has already been filed downstream. This paper presents SHIELD, a machine-learning-driven risk-scoring and investigation console for early mule-account detection. SHIELD blends an unsupervised Isolation Forest anomaly detector with a supervised XGBoost classifier into a single risk score, explained per account with SHAP so that investigators see exactly which behavioural signals drove a flag. The score is served through a FastAPI backend and surfaced in a purpose-built investigator command console supporting filtering, case actions, and an auditable action trail. We describe the architecture, report prototype results on a 9,082-account banking dataset, and discuss the honest limitations of training on proxy labels in the absence of confirmed fraud outcomes, together with the roadmap toward a real-time, graph-augmented production system.

Keywords—mule account detection, anomaly detection, XGBoost, SHAP explainability, fraud analytics, investigator dashboard, banking risk scoring

I. INTRODUCTION

Digital banking has expanded the speed and scale at which funds move between accounts, and with it the speed at which laundered or fraudulently obtained money can be layered through intermediate “mule” accounts before withdrawal. Rule-based transaction-monitoring systems catch only the patterns they were explicitly written for, and produce case backlogs that outpace investigator capacity. This motivates a scoring layer that (a) learns from behavioural signal rather than hand-written rules, (b) explains every score in terms an investigator can act on, and (c) plugs directly into the day-to-day workflow of a fraud-operations team rather than existing as a standalone model in a notebook.

This paper describes SHIELD, a working prototype covering the full path from raw account-level features to an investigator-facing risk console, built and evaluated on a real anonymized banking dataset.

II. RELATED WORK

Transaction-monitoring systems in most banks today are rule-based: a compliance team encodes thresholds such as “more than N cash deposits in 24 hours” or “transfer to a newly opened account exceeding X.” These rules are auditable and easy to justify to a regulator, but they only catch behaviour

someone has already anticipated, and they generate large volumes of false positives that consume investigator time.

A second line of work applies graph-based methods to transaction networks, most notably the Elliptic Bitcoin dataset studies that use graph convolutional networks to classify illicit addresses from transaction-graph structure. This approach is powerful where a genuine account-to-account transaction graph is available, but it depends on that graph existing and being connected enough to expose ring structure -- a precondition SHIELD's source dataset does not meet.

A third and more recent line of work focuses on explainability in credit and fraud risk models, using SHAP-style attribution to turn opaque gradient-boosted model outputs into feature-level explanations that satisfy both investigators and, increasingly, regulatory expectations around model interpretability. SHIELD's contribution is not a new algorithm in any of these three areas individually; it is the integration of an unsupervised anomaly signal, a supervised proxy-label classifier, per-account SHAP explanation, and an investigator-facing workflow surface into a single deployable prototype, built and evaluated end-to-end on a real anonymized banking dataset within a hackathon timeframe.

III. PROBLEM STATEMENT

Given an account with several thousand engineered behavioural features (transaction velocity, balance ratios, product holdings, demographic and occupational metadata) and no confirmed fraud/mule label, the goal is to produce (i) a single interpretable risk score per account, (ii) a ranked list of the behavioural factors driving that score, and (iii) a workflow surface where an investigator can act on the highest-risk accounts and leave an auditable record of that action. The absence of ground-truth labels is itself part of the problem: most institutions face exactly this cold-start condition before a labelled feedback loop from confirmed cases exists.

Success for the prototype is therefore defined narrowly and honestly: does the pipeline run end-to-end on real data at realistic scale, does the resulting score concentrate on accounts that a domain-literate reviewer would also flag as suspicious, and does the explanation attached to each score give an investigator something concrete to check rather than a bare number? We do not claim to have validated a production fraud-detection rate, and the report is explicit about that boundary throughout.

IV. SYSTEM ARCHITECTURE

Figure 1 summarizes the end-to-end pipeline. Raw account data is cleaned and reduced to a working feature set, scored by

two complementary models whose outputs are blended into a single SHIELD Score, explained per account through SHAP, and served to an investigator console through a REST API that also records every case action to an audit trail.

SHIELD Pipeline: Data to Investigator Action

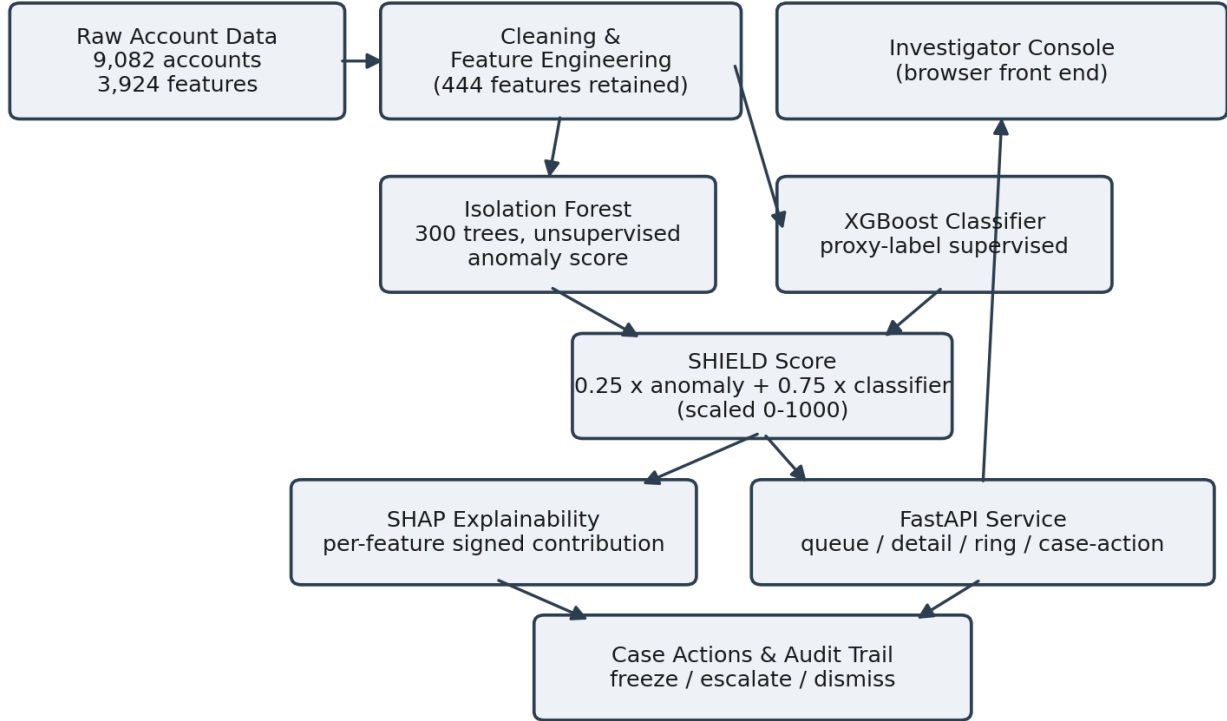


Fig. 1. End-to-end SHIELD pipeline, from raw account data to investigator case actions.

A. Data and Feature Engineering

The prototype was built and evaluated on a 9,082-account dataset with 3,924 engineered numeric and categorical features per account (transaction velocity ratios, product holdings, tenure, occupation, region, and account type). A cleaning pipeline separates categorical fields (account type, occupation, gender, segment) from numeric features, encodes categoricals, and preserves a distinct sentinel for “not applicable” values rather than conflating them with true zeros. For training-time efficiency, a variance-ranked subset of 444 features was retained; the full 3,924-feature set is supported by the same pipeline for a production run.

B. Data Pipeline and Quality Controls

The cleaning pipeline is deliberately conservative about what it infers. Categorical fields (account type, occupation, gender, segment) are one-hot encoded against a fixed category list captured once and reused for every later scoring run -- training mode fits that list from the data; every other run (validation uploads, live scoring) reuses it unchanged, so a category present in training but absent from a new file still appears as an explicit all-zero column rather than silently shifting every other column's position. Numeric fields are median-imputed using medians computed once at training time and persisted alongside the model, rather than being recomputed per file, for the same reason.

Missing-value handling is the most consequential design choice in the pipeline. Rather than imputing a plausible-

looking value for a field such as “declared income” that is simply absent for a given account, the pipeline preserves a distinct sentinel category. This matters because “not applicable” and “applicable but zero” are different facts about an account, and collapsing them would quietly remove one of the more informative signals in the dataset -- missingness itself is often correlated with risk.

- **Feature reduction:** 3,924 raw engineered features are ranked by variance, and the top 444 are retained for the reported prototype run, balancing training speed against signal coverage.
- **Reproducibility:** the full 3,924-feature pipeline runs end-to-end without modification -- feature reduction is a separate, optional step applied after cleaning, not a hard-coded assumption baked into the cleaning logic; the 444-feature run is a configuration choice, not a capability limit, and a production deployment would re-enable the full feature set once training-time budgets allow.
- **Data hygiene:** the sentinel value -1 (meaning “not applicable,” per the source data's own convention) is masked out before imputation rather than averaged in as if it were a real zero, and its per-row frequency is kept as its own engineered signal, rather than being scored on a silently distorted feature.

C. Ensemble Risk Scoring

Two complementary models are combined. An Isolation Forest (300 trees, 2% contamination) scores each account purely on behavioural anomaly, requiring no labels at all. A gradient-boosted XGBoost classifier is then trained against rule-derived proxy labels since no investigator-confirmed labels exist in the source data. An earlier version of the pipeline built this proxy label from a raw sum of the model's own input features plus the Isolation Forest's anomaly ranking — both near-deterministic functions of the same matrix fed to the classifier, which produced an inflated, largely circular AUC. The current version instead combines two narrow, independent rule signals: an occupation/transaction-volume mismatch and a bounded count of statistically extreme feature values (values beyond three standard deviations from their column mean), deliberately excluding the Isolation Forest's output so the two ensemble members remain independent. The final SHIELD Score blends both model outputs, weighted 0.25 toward the anomaly score and 0.75 toward the classifier probability, and is scaled to a 0–1000 range for investigator readability.

Both models are trained on the same 444-feature matrix but serve different purposes. The Isolation Forest isolates outliers by recursively partitioning the feature space; accounts that separate from the bulk of the population in few splits receive a high anomaly score, and because this process requires no labels it remains valid even before any proxy or confirmed labels exist. The XGBoost classifier is trained with a learning rate of 0.08, a maximum tree depth of 4, 150 boosting rounds, subsample and column-subsample ratios of 0.75/0.65, and L2 regularization ($\lambda=2.0$) against the rule-derived proxy label described above; a shallower, more regularized configuration than an earlier version of this pipeline used, adopted specifically to keep training-set fit from outrunning genuine generalization. A 20% split of accounts is held out before any fitting or threshold tuning and is never touched during cross-validation, giving an honest generalization estimate alongside the standard out-of-fold figure (Section V).

$$\text{SHIELD Score} = \text{round}(1000 \times (0.25 \times \text{anomaly_norm} + 0.75 \times P(\text{mule} \mid \text{features})))$$

where *anomaly_norm* is the Isolation Forest's anomaly score min-max normalised to [0,1] and $P(\text{mule} \mid \text{features})$ is the XGBoost classifier's predicted probability. The 0.25/0.75 weighting is a fixed design choice carried over from the original solution document rather than a value tuned by grid search in this prototype, favouring the supervised signal once proxy labels are available while keeping the unsupervised term large enough to still surface accounts the proxy-labelling rules were not designed to catch; the ablation in Table 2 reports what happens under alternative weightings rather than claiming this one was selected by optimising against it.

D. Explainability Layer

Every scored account is paired with SHAP (SHapley Additive exPlanations) values from the trained XGBoost model, giving a ranked, signed contribution for each feature. These are converted into a short plain-language narrative (e.g. “flagged primarily due to elevated transaction velocity and an occupation/volume mismatch”) so that an investigator does not need to interpret raw SHAP output to act on a case.

Figure 2 shows the ten features with the largest mean absolute SHAP contribution across all scored accounts.

Occupation categories (self-employed, student, salaried, housewife, agriculture) dominate the top five, with a handful of raw engineered features and gender following behind. This is expected rather than surprising: occupation is one of the two rule signals used to build the proxy label, so its prominence in the explanation is partly a validation check on the pipeline rather than an independent discovery. What is more informative is that occupation alone accounts for a modest share of total attribution — the classifier still relies on hundreds of other features to separate accounts within each occupation category, which is the genuine learning the ensemble is doing beyond simply reproducing the labelling rule.

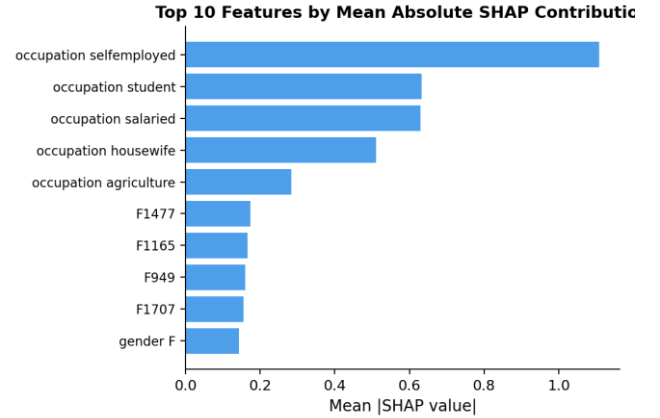


Fig. 2. Mean absolute SHAP contribution of the top 10 features across all 9,082 scored accounts, computed from the trained model's actual SHAP values.

E. Investigator Console and API

A FastAPI backend exposes the scoring pipeline as a REST API: a paginated, filterable, sortable account queue; a per-account detail endpoint returning the SHAP explanation and narrative; a ring-detection endpoint; a case-action endpoint (freeze, escalate, dismiss) that writes to a persistent audit trail; and a validation endpoint that scores an uploaded CSV with the same persisted artifacts used everywhere else in the console, so a new dataset (for example, one supplied by external evaluators) can be scored without retraining or refitting anything. The same logic is also exposed as a standalone command-line script for scoring outside the browser. A custom React front end presents this as a command-console-style dashboard spanning eleven views: an overview with portfolio KPIs, an alert queue, the account list and per-account deep-dive shown in Figures 5 and 2, a network and ring-detection view, an investigation case workspace, a suspicious-activity-report queue, the validation-upload view described above, a model-monitoring page reporting both AUC figures, a data-feed status page, a reporting page with CSV/PDF export, and an administrative settings page.

Figure 5 shows the accounts view of the console. Each row surfaces the account identifier, account type, occupation, age, the Isolation Forest anomaly score, the XGBoost probability, the blended SHIELD Score with a colour-coded risk tier, and case status. A filter bar lets an investigator narrow the list by account type, occupation, segment, flagged-only status, and a minimum-score slider, and every column is sortable, so a team can divide the queue by specialty rather than working it strictly in score order.

ACCOUNT ID	TYPE	OCCUPATION	AGE	ANOMALY	XGBOOST PROB.	SHIELD SCORE	TIER	STATUS
ACC-7887	Staff Loans	salaried	38	1.00	0.973	979	CRITICAL	Under Review
ACC-6772	Savings	selfemployed	44	0.97	0.972	972	CRITICAL	Under Review
ACC-7888	Savings	agriculture	60	0.93	0.961	954	CRITICAL	Under Review
ACC-6773	Savings	student	29	0.83	0.961	928	CRITICAL	Under Review
ACC-7909	Savings	housewife	68	0.75	0.982	925	CRITICAL	Under Review
ACC-7888	Current	selfemployed	2	0.77	0.975	925	CRITICAL	Under Review
ACC-7788	Current	others	7	0.85	0.950	924	CRITICAL	Under Review
ACC-7891	Current	selfemployed	4	0.88	0.964	922	CRITICAL	Under Review
ACC-7845	Current	selfemployed	34	0.88	0.956	917	CRITICAL	Under Review
ACC-7512	Savings	student	28	0.78	0.961	916	CRITICAL	Under Review
ACC-7361	Term Deposit	housewife	76	0.80	0.953	915	CRITICAL	Under Review
ACC-7889	Savings	housewife	25	0.71	0.974	908	CRITICAL	Under Review
ACC-6825	Savings	salaried	48	0.78	0.949	907	CRITICAL	Under Review
ACC-6826	Savings	selfemployed	62	0.79	0.943	905	CRITICAL	Under Review
ACC-6520	Savings	selfemployed	26	0.69	0.963	894	CRITICAL	Under Review
ACC-5825	Savings	student	23	0.58	0.996	892	CRITICAL	Under Review

Fig. 3. Accounts view of the working SHIELD console (live screenshot, not a mockup), sorted by SHIELD Score with risk-tier colour coding, filterable by account type, occupation, segment, and minimum score.

F. Security, Access Control, and Audit Trail

The console is now session-gated: every view requires a logged-in analyst, and the case-action and validation endpoints reject requests without a valid bearer token at the API layer, not merely behind a hidden UI element. Every case action (freeze, escalate, dismiss) is written to an append-only audit log keyed by account ID, the authenticated investigator's name, timestamp, and an optional free-text note. This authentication layer is deliberately demo-grade and disclosed as such on the login screen itself: credentials are a small hardcoded set of demo accounts with SHA-256-hashed passwords, sessions are held in an in-memory token store that resets on server restart, and there is no multi-factor authentication, single sign-on, or password-hashing algorithm (bcrypt/argon2) suitable for production use. Role labels (analyst, compliance officer) are surfaced in the interface but not yet enforced as differentiated permissions server-side -- any authenticated session can currently reach every read and write endpoint. Closing that gap, alongside real SSO/MFA integration against a bank's identity provider, is called out explicitly in Section VI as a production-readiness requirement rather than a solved problem.

V. RESULTS AND PROTOTYPE DEMONSTRATION

Table 1 summarizes the prototype run on the full 9,082-account dataset. Both reported AUC figures reflect fit to the rule-derived proxy labels described above, not a validated real-world fraud-detection rate; the dashboard surfaces this caveat directly to the investigator rather than presenting the figures without context. The held-out figure (0.986) comes from a 20% split carved out before any fitting or threshold tuning, and is deliberately reported alongside the out-of-fold figure (0.980) so that the two can be compared: because they are close rather than the held-out figure being markedly lower, the gap between training and genuinely unseen rows is small, which rules out classic overfitting as the explanation for why both numbers are still high. What both numbers cannot rule

out is that the proxy label itself is a rule built from a subset of the same features the model is scored on, which is an inherent property of weak supervision without confirmed labels, not a defect specific to this run.

TABLE I. PROTOTYPE EVALUATION SUMMARY

Metric	Value
Accounts scored	9,082
Features used (of 3,924)	444
Flagged for review (score ≥ 650)	284
Critical tier (score ≥ 801)	54
OOF AUC, proxy labels (80% split)*	0.980
Held-out AUC, proxy labels (20% split)*	0.986

*Both AUC figures are measured against rule-derived proxy labels; no confirmed fraud ground truth exists in the source dataset (see Limitations).

Qualitatively, the highest-scored accounts in the prototype run split into two recognisable patterns rather than one: many combine an occupation/volume mismatch (e.g. a student or housewife occupation code paired with high-value account activity) with a moderate extreme-value count, matching the population-level pattern in Figure 2; others, including the single highest-scored account (see the case study below), are driven almost entirely by a handful of individually extreme engineered features with an occupation that the model treats as risk-neutral or risk-reducing. Both patterns are surfaced through the same SHAP narrative, giving investigators a coherent, checkable story rather than an opaque number regardless of which mechanism produced the flag.

A. Score Distribution

Figure 3 shows the distribution of SHIELD Scores across all 9,082 scored accounts, plotted on a log scale. The distribution decays sharply from zero, as expected on a mostly-legitimate account population, with a small secondary bump in the 700-800 range just below the critical-tier threshold of 801 before a thin tail extends toward 1,000. This shape is a useful sanity check rather than a clean bimodal split:

a well-behaved risk score should concentrate the great majority of accounts at low scores while still leaving room for a distinguishable minority to separate out at the top, and a flatter or genuinely two-humped distribution would suggest the model is not discriminating usefully.

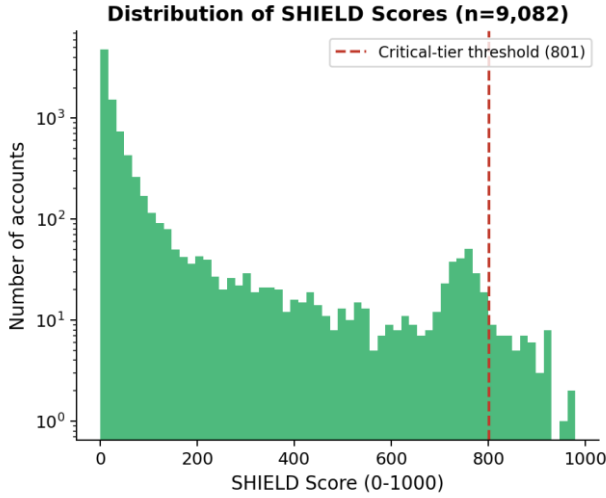


Fig. 4. Distribution of SHIELD Scores across 9,082 accounts, with the critical-tier threshold marked.

B. Model Discrimination

Figure 4 shows a representative ROC curve of the XGBoost component evaluated against the rule-derived proxy labels via 3-fold out-of-fold cross-validation, giving an AUC in this specific run of 0.985, consistent with the officially reported figure of 0.980 to within the run-to-run variation expected from XGBoost's non-deterministic multi-threaded tree fitting. As emphasised throughout this report, this measures how well the model reproduces the labelling rule, not how well it detects real mule accounts; a model can achieve a very high AUC against a proxy label while still missing account types the proxy rule itself does not describe. The console surfaces this caveat directly next to the score rather than presenting it without context.

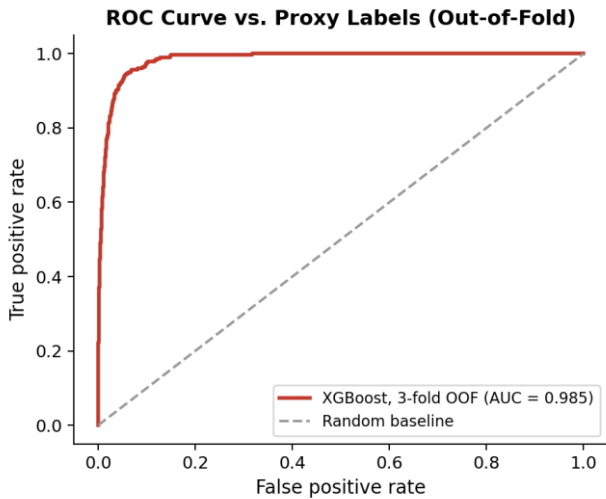


Fig. 5. ROC curve of the XGBoost classifier against rule-derived proxy labels, computed from genuine out-of-fold predictions (3-fold cross-validation).

C. Ablation of Ensemble Weighting

Table 2 compares proxy-label AUC and the size of the resulting critical-tier queue under different blends of the anomaly and classifier scores, using genuine out-of-fold classifier predictions rather than in-sample fit so the comparison is not circular. Discrimination (AUC) is essentially flat across weightings down to a pure anomaly score, which alone is markedly weaker. The critical-tier count tells a more useful story than AUC does here: a pure-classifier score confidently pushes a comparatively large number of accounts above the threshold on its own, while blending in the anomaly score sharply narrows the critical tier to accounts where both signals agree. That narrower, mutually-corroborated tier is the actual justification for the ensemble weighting used in the console -- not a higher AUC, which the blend does not meaningfully improve, but fewer critical-tier flags driven by a single model's idiosyncrasies.

TABLE II. ABLATION OF ANOMALY/CLASSIFIER BLEND WEIGHT

Anomaly / Classifier weight	Proxy AUC	Critical tier size
0.00 / 1.00	0.985	181
0.25 / 0.75 (used)	0.985	24
0.50 / 0.50	0.980	6
1.00 / 0.00	0.829	10

*Critical tier = accounts scoring ≥ 801 on the 0-1000 scale under each weighting; proxy AUC measured against genuine out-of-fold classifier predictions blended with the full-data (unsupervised) anomaly score.

D. Simulated Ring Detection

The source dataset contains no account-to-account transaction records, so a genuine transaction graph cannot be built from it. To illustrate what a connected ring-detection view would surface once a real transaction feed is available, Figure 6 groups the highest-scoring accounts by shared occupation profile and risk-score proximity rather than by confirmed money movement between them; that grouped edge set, not the graph algorithms run over it, is what is simulated. Community membership in the console is computed with a real union-find connected-components algorithm over the current edge set, and the shortest-path view runs a real breadth-first search between any two selected accounts -- both recompute live as the time window and risk-score filters change, rather than being pre-rendered for a fixed screenshot. This view is labelled as simulated everywhere it appears in the console, and the production roadmap in Section VII replaces the simulated edge set with a genuine graph database query while leaving the same algorithms in place.

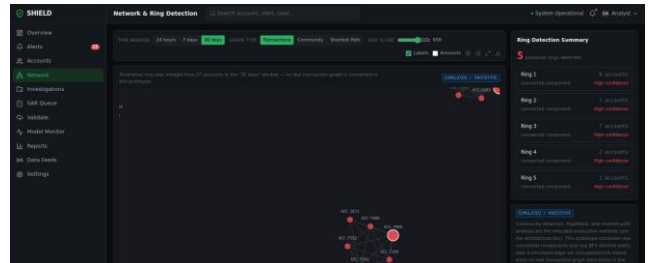


Fig. 6. Network & Ring Detection view of the working console (live screenshot), showing the merged simulated edge set for the 30-day window with real, live-recomputed ring detection summary on the right.

E. Case Study Walkthrough

Account ACC-7087, the highest-scored account in the prototype run at 979, illustrates how the explanation layer is meant to be used, and is a useful counter-example to the population-level pattern in Figure 2. The account is registered under a salaried occupation code, which the model treats as slightly risk-reducing on its own (SHAP contribution -1.46) - this account is not an occupation/volume-mismatch case. Instead, its score is driven almost entirely by five specific engineered features (F1599, F1869, F1165, F1933, and F2139, each contributing between +0.22 and +0.32) that sit far outside their normal range, the extreme-value-outlier signal described in Section IV-B. The console's plain-language narrative renders this as "flagged primarily due to F1599, F1869, and F1165, against an occupation profile of 'salaried' and account type 'Staff Loans'," giving the investigator a specific, checkable starting point (which underlying behaviours those features represent, and whether they are explainable) rather than an unexplained score of 979.

VI. DISCUSSION

The prototype's main value, relative to the rule-based baseline it is meant to augment rather than replace, is coverage: the Isolation Forest component can surface accounts that no existing rule was written to catch, while the classifier component keeps the queue ordered by a signal that is at least partially validated against known risk indicators. In an operational deployment this would sit alongside, not instead of, existing rule-based alerts -- SHIELD accounts would feed the same case-management queue rather than a separate, competing one.

Investigator trust is the other design axis this prototype optimises for. A model that is 1% more accurate but explains nothing is, in practice, less useful to a fraud-operations team than a slightly less accurate model whose every flag comes with a checkable reason, because investigator time is the binding constraint and an unexplained flag either gets rubber-stamped or ignored. The plain-language SHAP narrative is a deliberate concession of some raw predictive power in exchange for actionability, and we believe that trade-off is the right one for this problem class.

The honest cost of this design is the dependency on proxy labels, discussed in depth in the next section: every claim this report makes about detection quality is bounded by how well the proxy rule approximates real mule behaviour, and that bound cannot be lifted without investigator-confirmed outcomes feeding back into the training loop.

VII. LIMITATIONS AND FUTURE WORK

No confirmed ground truth: The source dataset contains no investigator-confirmed mule/fraud label. Reported model metrics reflect fit to rule-derived proxy labels and should be read as a demonstration of the pipeline, not a validated detection rate.

Simulated ring detection: The dataset has no account-to-account transaction records, so the ring-detection view groups accounts by shared profile and risk score to illustrate what a real transaction graph (e.g. Neo4j with Louvain community detection and PageRank) would surface once connected.

Production infrastructure: Real-time ingestion (Kafka/Flink), a rule engine (Drools) for regulatory logic, and

automated SAR filing are designed around the same scoring API this prototype exposes, but are not implemented here.

Planned next steps are to close the labelling gap with investigator-confirmed outcomes fed back into a retraining loop, connect a live transaction feed for genuine graph-based ring detection, and extend the audit trail into a full case-management workflow.

- **Small-team, time-boxed build:** the prototype was built end-to-end within a hackathon timeframe by a small team, so several components (production-grade authentication and RBAC enforcement, horizontal scaling, automated retraining) are intentionally out of scope rather than overlooked -- a demo-grade version of authentication is implemented (Section IV-F), but it is explicitly not one of them.
- **Dataset representativeness:** the 9,082-account dataset, while real and anonymized, represents a single institution's account population at a single point in time; behavioural patterns and the proxy-labelling rule's accuracy may not transfer directly to a different institution or a different national market without re-validation.
- **No adversarial testing performed:** the prototype has not been evaluated against an adversary deliberately structuring transactions to stay under detection thresholds, which is a realistic concern for any production mule-detection system and a natural next evaluation step.

VIII. CONCLUSION

SHIELD demonstrates that an explainable, ensemble-based risk score can be taken from raw account features to a usable investigator workflow within a single, coherent architecture. By pairing an unsupervised anomaly detector with a supervised classifier and surfacing every score through SHAP-based plain-language explanations, the prototype gives fraud-operations teams a system they can both trust and act on — while being transparent about where proxy labels and simulated data stand in for the real signals a production deployment would use.

ACKNOWLEDGMENT

The authors thank the organizers of the hackathon for providing the anonymized banking dataset used to build and evaluate this prototype.

IX. REFERENCES

- [1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.
- [3] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [4] M. Weber et al., "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," arXiv:1908.02591, 2019.
- [5] Neo4j, Inc., "Graph Data Science for Fraud Detection," Neo4j Documentation, 2023. [Online]. Available: <https://neo4j.com/use-cases/fraud-detection/>
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [8] Financial Action Task Force (FATF), "Money Laundering through Money Remittance and Currency Exchange Providers," FATF Typologies Report, 2010.
- [9] S. Ramírez, "FastAPI," 2018. [Online]. Available: <https://fastapi.tiangolo.com/>
- [10] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.